

Revisit DANE for the Web: A Performance Perspective

Hyeonmin Lee
Hanyang University
Ansan, Republic of Korea
hyeonminlee@hanyang.ac.kr

Yixin Sun
University of Virginia
Charlottesville, Virginia, USA
ys3kz@virginia.edu

Abstract

DNS-based Authentication of Named Entities (DANE) offers an alternative or complementary authentication mechanism. However, DANE's web adoption is limited. In this work, we revisit DANE by focusing on a practical deployment concern for the web: added latency. Using our custom DANE-for-the-web measurement framework, we show that DANE's overhead is dominated by DNS resolution, particularly DNSSEC, which is highly sensitive to network latency and caching behavior. We further reveal that certain TLSA parameter combinations can trigger DNS-over-TCP fallback due to large response sizes, increasing DNS delay by up to 2 round trip times. Finally, our web-scale study discovers 4,435 websites supporting DANE validation for HTTPS connections, and characterizes DANE-induced delays across these real-world domains. Our findings shed light on the readiness of DANE for the web and offer guidance for efficient deployment.

CCS Concepts

• **Networks** → **Network measurement**; **Network experimentation**; • **Security and privacy** → **Security protocols**.

Keywords

Measurement, DNS, DNSSEC, DANE, Web

ACM Reference Format:

Hyeonmin Lee and Yixin Sun. 2026. Revisit DANE for the Web: A Performance Perspective. In *Proceedings of the 2026 ACM Internet Measurement Conference (IMC '26)*, October 12–16, 2026, Karlsruhe, Germany. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3777912.3839830>

1 Introduction

Transport Layer Security (TLS) authentication relies on Certificate Authorities (CAs), but past compromises and mis-issuances [3, 8, 27, 29] have exposed structural weaknesses in this trust model. The DNS-based Authentication of Named Entities (DANE) protocol [12, 17] enables domain owners to bind their public keys (or TLS certificates) directly to domain names by leveraging the Domain Name System (DNS) secured with DNS Security Extensions (DNSSEC) [36–38]. By doing so, DANE can reduce reliance on (trusted) CAs and provide a more domain-controlled authentication model.

Currently, DANE is mainly deployed in the email ecosystem, where email servers authenticate peer servers using DANE, which

offers security benefits such as mitigating TLS downgrade attacks. In contrast, DANE deployment for the web remains very limited, and one concern is that DANE validation can introduce additional latency for delay-sensitive web services [18, 23]. However, the degree and contributing factors of such delay have not been well studied.

In this study, we perform an in-depth analysis of the delays introduced when DANE is adopted for the web. We develop a *DANE-for-the-web* measurement framework, which decomposes web page load delays into detailed components (e.g., DNS resolution and TLS handshake), considers DNS caching effects, and examines variations across different DANE configurations and client locations. Using this framework, we conduct controlled experiments using our own DNS and web servers. We further perform Internet-wide scans and discover 8,513 domains serving TLSA records on port 443 (common port for HTTPS services). We then evaluate real-world delays with DANE by connecting to these domains.

Key Contributions. Our key findings/contributions are:

- We build a DANE-for-the-web measurement framework by extending an existing DANE proxy to support all TLSA configurations and enable detailed delay measurement.
- The dominant delay component stems from DNSSEC resolution, which grows significantly with the Round Trip Time (RTT) to the authoritative nameserver.
- In most favorable conditions (e.g., warm cache and low RTT), the performance benefit of DANE validation can offset the additional resolution delay of TLSA records and even slightly outperform standard TLS validation.
- Certain TLSA configurations (e.g., TLSA 000) result in substantial overhead due to UDP fragmentation of the larger DNS responses and consequently TCP fallback.
- We successfully perform DANE validation in connections to 4,435 websites. DANE adds a median overhead of 42.4 ms in Europe and 744.0 ms in US East when using Google public resolver.

Artifact availability. Our code and dataset, including our DANE-for-the-web measurement framework, are publicly available at:

<https://github.com/dane-web-study/dane-web-study>

2 Background and Related Work

With DANE [12, 17], a domain can publish its public key or certificate as a TLSA record, a DNS record type for DANE.

TLSA records provide information that can be used to verify the public key or certificate of a service running on a domain. Because multiple services can share the same domain on different ports, each TLSA record is stored under a prefixed domain name that includes the port (e.g., 443) and protocol (e.g., TCP). A TLSA record consists of four fields:



This work is licensed under a Creative Commons Attribution 4.0 International License. *IMC '26, Karlsruhe, Germany*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2327-8/2026/10

<https://doi.org/10.1145/3777912.3839830>

- **Certificate Usage:** specifies how the presented certificate (chain) should be verified, with four options: PKIX-TA (0), PKIX-EE (1), DANE-TA (2), and DANE-EE (3). Both PKIX usages require certificates issued by trusted CAs and validation through the existing root store [2, 30, 32]. PKIX-TA validates a CA certificate (i.e., TLSA record contains the CA's certificate), while PKIX-EE validates the end-entity certificate (i.e., TLSA record contains the server's leaf certificate). In contrast, both DANE usages do not require trusted CA-issued certificates; self-signed or internally issued certificates can be used instead. DANE-TA validates a Trust Anchor (TA)'s certificate (e.g., a non-public CA used in a corporate network) without requiring root store verification, while DANE-EE directly validates an end-entity certificate (possibly self-signed).
- **Selector:** specifies which part of the certificate is used for Certificate Association Data; the entire certificate (0) or only the public key part (1).
- **Matching Type:** specifies the format of Certificate Association Data; the original data (0), its SHA-1 hash (1), or its SHA-2 hash (2).
- **Certificate Association Data:** contains the data generated according to the above three parameters.

DNSSEC [36–38] guarantees the authenticity and integrity of DNS records. Domains that deploy DANE *must* support DNSSEC to protect their TLSA records from attacks such as cache poisoning. A domain (e.g., anon-test.com¹) should publish DNSKEY and RRSIG records in its authoritative nameserver and upload a corresponding DS record to its parent zone (e.g., .com) to establish a DNSSEC chain of trust. Clients typically rely on recursive resolvers for DNSSEC validation.

Client-side DANE validation consists of two phases: (1) the *DNS phase* and (2) the *TLS handshake phase (TLS phase)*, as illustrated in Figure 8 in Appendix B.

- (1) **DNS phase:** DNS lookup and DNSSEC validation are carried out ([i]–[iii]), prior to initiating the TLS handshake. The browser sends DNS queries for TLSA records of the target domain to its DNSSEC-enabled resolver ([i]), which performs recursive lookups ([ii]) and returns the TLSA together with their DNSSEC validation results ([iii]), indicated by the AD flag [42]. If the browser receives a TLSA record with AD = 1, it proceeds to the TLS phase. Otherwise, if DNSSEC validation fails (AD = 0), the browser aborts the connection (or may fall back to standard TLS, depending on its configuration). The TLSA lookup can be performed in parallel with the A record lookup. Without DANE (e.g., in standard TLS), only A records are retrieved ([a]–[c]).
- (2) **TLS phase:** During the TLS handshake ([iv]–[vi]), the browser verifies the server's certificate using the retrieved TLSA records (from the *DNS phase*). If verification fails, the connection is aborted. In standard TLS, this phase ([d]–[f]) involves validating the certificate chain using the trusted root store [2, 30, 32].

Further background and TLSA examples are in Appendix B.

Related work. While several studies have examined DANE for email services [24, 25], there is very limited work on DANE for

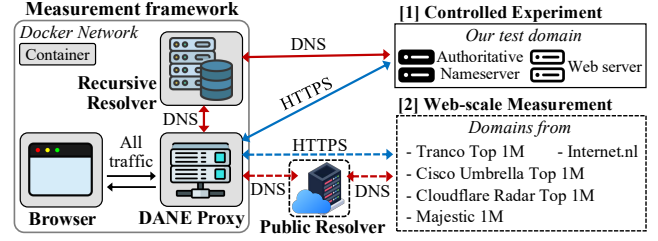


Figure 1: Overview of our measurement framework.

the web. In 2015, Zhu et al. [41] conducted Internet-wide scans to examine DANE configurations in HTTPS, SMTP, and XMPP protocols. Most recently, Yagi et al. [40] measured web page load times with and without DANE.

Our work differs from [40] in three main aspects. (1) *Measurement framework and delay analysis:* [40] uses the original Let's DANE proxy [5], which supports only DANE-EE, and routes only DANE-enabled webpage loads through the proxy. In contrast, we extend the proxy to support all TLSA parameters and use the same proxy path for webpage loads both with and without DANE. Moreover, we add detailed logging to decompose the delay into DNS and TLS phases. (2) *Resolver and DNS cache configuration:* [40] uses a local Unbound recursive resolver and considers cache-disabled and warm-cache settings. Our controlled experiments explicitly distinguish caching-disabled, cold-cache, and warm-cache conditions, while our web-scale experiment uses Google Public DNS (without direct control over its cache state). (3) *Dataset and TLSA scope:* [40] evaluates 4,022 TLSA-enabled domains obtained solely from Internet.nl [20] and considers only DANE-EE configurations. In contrast, our web-scale measurement discovers 8,513 TLSA-enabled domains from multiple datasets and evaluates their deployed TLSA configurations, including not only DANE-EE but also other TLSA parameter combinations. Furthermore, our controlled experiments systematically evaluate all possible TLSA parameter combinations.

• **Takeaway.** These analyses enable us to investigate *when* and *why* DANE introduces additional latency, rather than only its impact on end-to-end webpage load time. In particular, our analysis identifies DNSSEC resolution as the dominant source of DANE-induced delay, quantifies the effects of RTT and resolver caching, and identifies DNS-over-TCP fallback caused by oversized TLSA responses as a configuration-dependent source of additional latency.

3 Measurement Framework

We develop a *DANE-for-the-web* measurement framework that integrates three modules, as illustrated in Figure 1.

1) Web browser. We use Firefox (v140.0.2 [31]) in headless mode via Selenium. The browser runs inside a dedicated Docker container, and is used to measure page load time.

2) DANE proxy. Since mainstream browsers do not support DANE-based TLS authentication, we use Let's DANE [5], a proxy that enables DANE validation for client applications such as browsers. With the proxy, all HTTPS requests from the browser are forwarded to the proxy, which performs DNS resolution, initiates the TLS handshake, and returns the response. When DANE is enabled (i.e., DANE-TLS mode), the proxy performs a TLSA lookup (in parallel with the A lookup) and conducts the TLS handshake using DANE. If

¹A non-existent domain name used for anonymization.

DANE is disabled (i.e., STD-TLS mode), the proxy performs the standard TLS handshake by validating the certificate chain using the root store in Ubuntu 22.04. In both modes, communication between the browser and the proxy is encrypted with a custom certificate dynamically generated by the proxy. The proxy runs in a separate Docker container. Let's DANE, however, has several limitations that hinder comprehensive evaluation; for instance, prior work [40] was restricted to the DANE-EE usage because the proxy only supports that one usage type. To tackle this limitation, we extend Let's DANE to support all TLSA parameters and ensure consistent behavior across both STD-TLS and DANE-TLS modes. We further build a logging module to record detailed delay components. In total, 553 lines of code are changed.

3) Recursive resolver. We use the official BIND9 (v9.20) Docker image from ISC [21], running in a separate container, for the controlled experiments. For the real-world websites, we use a public resolver instead of our own resolver.

3.1 Measurement Metrics and Scenarios

3.1.1 Delay Metrics. Our framework measures:

Page load time. We use Firefox's 'window.performance.getEntries()' API and calculate the loading time as 'loadEventEnd' - 'start-time'. This represents the end-to-end webpage load time measured in our browser setup.

The detailed delays are measured by our DANE proxy:

DNS phase. In DANE-TLS mode, the delay between generating and sending TLSA and A queries (in parallel) to the resolver container and receiving the corresponding responses is measured. This delay includes DNSSEC validation time. In STD-TLS mode, only the A lookup time is measured.

TLS phase. In DANE-TLS mode, the proxy measures the TLS handshake delay, which varies depending on the TLSA parameter combinations. In STD-TLS mode, only the certificate verification delay via the trusted root store is measured (e.g., 'w/o DANE' in Figure 7 in Appendix B).

3.1.2 Test Scenarios. We consider three main scenarios:

- (1) **Baseline:** A baseline scenario where the DANE proxy is not used. The browser connects directly to our domain.
- (2) **STD-TLS:** The standard TLS scenario, where the browser connects to our domain through the DANE proxy but performs standard TLS authentication.
- (3) **DANE-TLS:** The DANE-enabled TLS scenario, where the browser connects to our domain through the DANE proxy and performs DANE-based TLS authentication.

For each main scenario, we further vary the recursive resolver configurations in terms of DNSSEC support and caching behavior to cover representative practical cases.

Resolver DNSSEC support. As some resolvers may not perform DNSSEC validation in practice, our framework supports two modes:

- **DNSSEC-non-validating (DNSSEC X):** The resolver does not perform DNSSEC lookups and validation.
- **DNSSEC-validating (DNSSEC O):** The resolver performs DNSSEC lookups/validation, returning the result (AD flag) to the proxy. Note that DANE-TLS must use the DNSSEC-validating configuration, as DNSSEC is essential for DANE.

Resolver cache. Our framework supports three cache modes:

- **Caching-disabled:** The resolver disables caching; the maximum Time-to-Live (TTL) values for caches are set to zero. This reflects a *worst-case* scenario in which the client uses a resolver without caching support, so no records are stored even within a single test, resulting in the maximum possible DNS lookup delay.
- **Cold cache:** Resolver caching is enabled but no relevant record is cached initially, representing a client accessing a rarely visited domain.
- **Warm cache:** Caching is enabled and all relevant DNS records are already cached and immediately reusable. This setup represents a *best-case* scenario in which the target domain has been visited recently. In this test, the browser first connects to our domain and then immediately reconnects; the second connection is used for analysis.

4 Controlled Experiment of DANE Delay

We conduct controlled experiments to measure the overhead introduced by adopting DANE for the web (Figure 1-[1]).

4.0.1 Testbed. We first build a controlled testbed.

Nameserver/Web server. We run an authoritative nameserver for a test domain (anon-test.com) using BIND9, which supports DNSSEC and publishes TLSA records generated from our web server's certificate, covering all 24 parameter combinations. Each subdomain corresponds to one parameter combination, with TTLs set to 1 second (Caching-disabled), 5 seconds (Cold cache), and 3,600 seconds (Warm cache); these are well below our resolver's maximum cache TTL (86,400s). Note that our resolver is configured correspondingly for each caching scenario, and the purpose of setting TTLs is to control unintended caching at potential intermediate entities/middleboxes in the path. Our web server runs Apache2 (v2.4.65) with a certificate chain issued by Let's Encrypt and serves a minimal single-line webpage. To support both STD-TLS and DANE-TLS experiments, we use a trusted CA-issued certificate instead of a self-signed one. Server configuration is identical for STD-TLS and DANE-TLS, except for DANE-EE where only the leaf certificate is served.

Controlling DNS cache effects. We initialize *fresh* Docker containers for all components before each run and remove them after completion. Furthermore, our nameserver dynamically generates a unique subdomain (*u-id.anon-test.com*) for each run and serves corresponding DNS records (i.e., A and TLSA). The browser sends queries to these unique subdomains to ensure that every lookup triggers a fresh resolution. We also flush the OS cache before each run.

EDNS0 setting. We configure both the DANE proxy and the recursive resolver to advertise a 4,096-byte EDNS0 UDP payload size in DNS queries.

Deployment location. We deploy our measurement framework in two AWS EC2 regions: US East and Europe (Germany). The name- and web server each run on separate EC2 instances in the US East region.

4.1 Experiment Results

By combining all scenarios and TLSA combinations, our experiment covers 84 distinct test cases. In each run, all 84 cases are executed

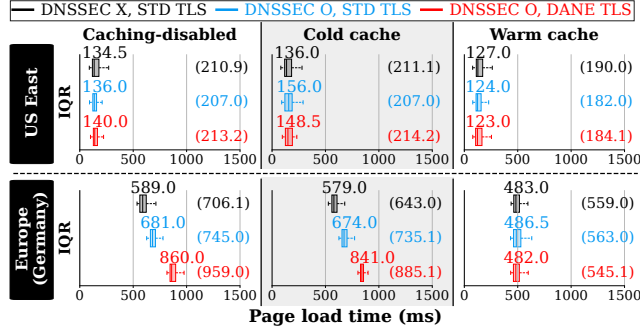


Figure 2: Interquartile range (IQR) box plots with median values for page load times (unit: ms). Values in parentheses denote the 95th percentile (p95).

sequentially, and the entire run is repeated 300 times to ensure statistical stability. The DANE proxy adds 48 ms of page load time on average across configurations and regions, compared to the baseline without the proxy; detailed results are in Appendix C.

4.1.1 STD-TLS vs. DANE-TLS. We first compare STD-TLS and DANE-TLS, where DANE-TLS uses the most common TLSA configuration (Certificate Usage 3, Selector 1, Matching Type 1), as reported in prior work [40] and observed in our web-scale measurements (Section 5). Note that all reported values refer to the median used in the IQR plots.

Page load time. Figure 2 shows page load times across different scenarios (e.g., DNSSEC support, caching behavior, and the use of DANE) in the two deployment regions. In US East, the overhead of enabling DANE is small: less than 10 ms across all scenarios, and only 4 ms when the resolver already performs DNSSEC validation (e.g., DNSSEC O, STD-TLS vs. DNSSEC O, DANE-TLS; caching-disabled). In contrast, all Germany scenarios show substantially larger page load times. The difference between STD-TLS and DANE-TLS is much more pronounced: in the worst case (caching-disabled), DANE increases the median load time by 271 ms, and decreases to roughly 262 ms with cold cache. However, under warm cache, DANE slightly *reduces* the load time by 1–4 ms compared to STD-TLS (in both US East and Germany). Next, we break down the delay components behind this difference.

DNS and TLS phases. Figure 3 shows the detailed delays.

(a) **US East:** shows only minor overhead when DANE is enabled. In the worst case (caching-disabled), it increases the *DNS+TLS* phase by about 11 ms relative to the ‘DNSSEC X, STD-TLS’ case, and by only 2 ms when the client already uses a DNSSEC-validating resolver (‘DNSSEC O, STD-TLS’). Interestingly, in the warm-cache setting, DANE-TLS becomes slightly faster (by about 3 ms) than both STD-TLS cases.

The middle row shows that the additional cost comes primarily from the *DNS* phase due to DNSSEC, especially in the caching-disabled and cold-cache settings. In contrast, the *TLS* phase is faster with DANE-TLS (about 6.1 ms for STD-TLS vs. 2.7 ms for DANE-TLS) because CA chain validation is not required. This reduction offsets the added *DNS* phase cost. In warm-cache (where *DNS* phase is similar across modes), the shorter *TLS* phase makes DANE-TLS slightly faster overall.

(b) **Germany:** shows different patterns due to the *longer* RTT between the measurement framework and our nameserver (in US East); while the RTT is about 1 ms from the framework in US East, it is roughly 91 ms from Germany, amplifying delays in both phases. This effect is particularly visible in the *DNS* phase. In the caching-disabled setting, enabling DNSSEC increases *DNS* phase by nearly 100 ms, and DANE adds about 190 ms. In the *TLS* phase, the trend remains the same as US East: DANE is about 3 ms faster than STD-TLS. Overall, because the *DNS* phase dominates the total delay, DANE results in about 280 ms of additional delay in the caching-disabled and cold-cache settings. However, in the warm-cache setting, DANE-TLS becomes slightly faster than STD-TLS for the same reason as in US East.

Takeaway. The main bottleneck for DANE is *DNS* phase, which is highly sensitive to network conditions such as RTT. However, when domains are frequently visited and in resolver caches (i.e., warm cache), DANE becomes slightly faster than STD-TLS due to faster validation in *TLS* phase.

4.1.2 TLSA parameters. We next examine how different parameter choices affect delay. When we fix Selector and Matching Type to 1 (their most common values, as shown in later Section 5), and vary only the Certificate Usage parameter, all four Usage types (011, 111, 211, 311) produce nearly identical page load times across regions and cache settings, with only marginal differences.

A more interesting pattern emerges when examining all combinations of Selector and Matching Type. Most combinations behave similarly, except for PKIX usages that embed the *full certificate*: TLSA 000 and 200. Figure 4a shows the page load time in Germany for TLSA Usage 0 (PKIX-TA) and 2 (DANE-TA), where 000 and 200 add roughly 200 ms compared to other parameters. Apart from these two cases, all combinations show nearly identical delays.

This behavior directly correlates with TLSA response sizes. As shown in Figure 4b, responses exceeding the common UDP fragmentation threshold (1,232 bytes [11]) trigger DNS-over-TCP fallback. In our measurements, all 1,273-byte responses, corresponding to the 000 and 200 combinations, always fall back to TCP. This fallback adds roughly 2 RTT of additional delay: a failed UDP attempt (≈ 1 RTT) followed by a TCP retry (≈ 1 RTT) and subsequent DNS lookup (≈ 1 RTT) together amount to ≈ 2 RTT. Other full certificate combinations (e.g., 100 and 300) do not exhibit this behavior in our setup because their certificates are smaller (1,096 bytes), keeping the response below the fragmentation threshold. However, if their certificate sizes grow beyond this limit, they would similarly incur TCP fallback. The complete plots for all parameter combinations are provided in Appendix D.1.

Takeaway. While most TLSA parameter combinations incur similar delays, combinations that embed full certificates (e.g., TLSA 000) introduce substantial overhead due to oversized DNS responses that trigger TCP fallback. This effect is especially amplified under high RTT conditions.

5 Web-Scale Measurement of DANE Delay

We perform large-scale scans to discover DANE-capable domains. We then measure the delay when connecting to these real-world domains in a practical setting (Figure 1–[2]).

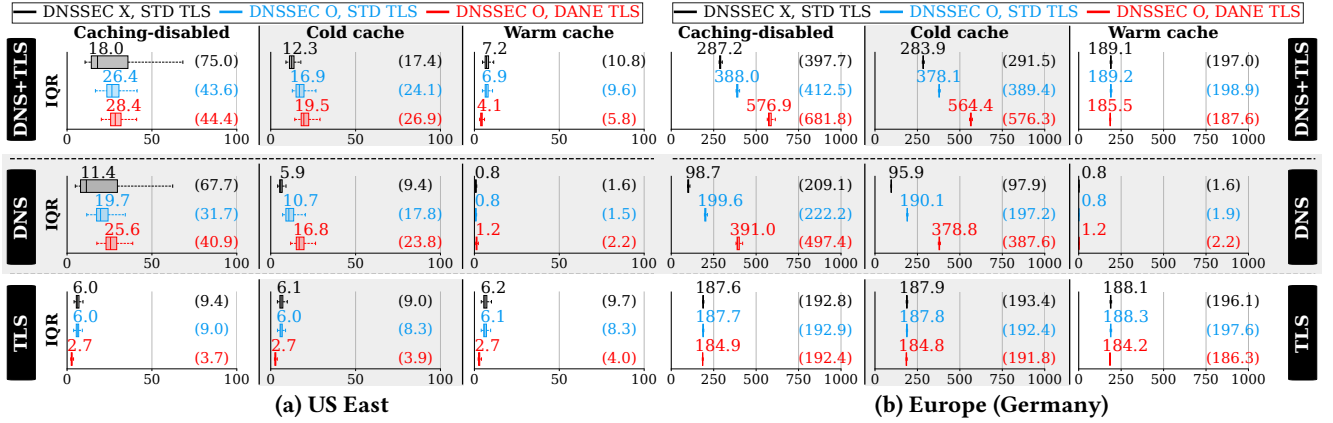


Figure 3: Breakdown of DNS+TLS, TLS, and DNS phase delays (unit: ms). Values in parentheses: 95th percentile.

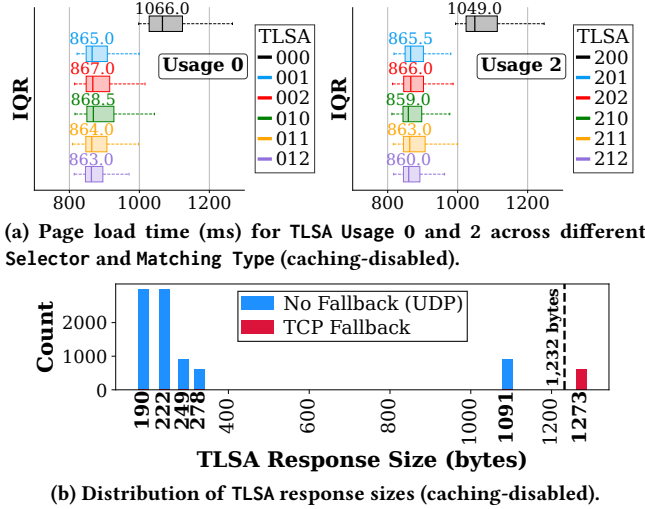
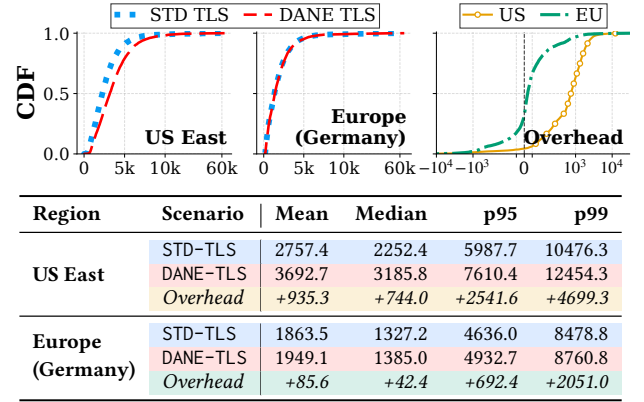


Figure 4: Page load times and TLSA response sizes for Germany. Trends are consistent in US East.

Target domains. We identify DNSSEC-enabled domains using the OpenINTEL datasets [34, 39], which continuously measure domains in four popularity rankings (Tranco [35], Cisco Umbrella [6], Cloudflare Radar [7], and Majestic Million [28]). From the datasets, we extract all domains that publish DNSKEY records. We additionally include 79,320 DNSSEC-enabled domains from internet.nl’s Hall of Fame [20]. As of April 29, 2026, this yields 240,429 unique FQDNs with DNSKEY. Note that OpenINTEL performs TLSA scans only for email ports (25, 465, 587); therefore, we perform our own TLSA lookups for these FQDNs on port 443 using ZDNS [22].

Domains with TLSA. We discover 8,513 domains with TLSA records for HTTPS (port 443), about 3.5% of the DNSKEY-publishing FQDNs. Among them, 84% use Usage 3 (DANE-EE). We also find 15 domains using TLSA 000 (*full-certificate* values). For two of these domains, we confirmed that they publish full-certificate TLSA values. Full statistics of usage types are in Appendix E.1.

Connection results. To reflect a realistic scenario, we use Google DNS (8.8.8.8), which covers over 60% of clients (that use open resolvers) in the wild [1], instead of our own resolver. Since Google

Figure 5: CDFs and statistics of page load times for web-scale measurements (unit: ms). *Overhead*: DANE-induced delay over STD-TLS.

DNS supports DNSSEC validation and caching, we only measure STD-TLS and DANE-TLS modes. We measure each domain in five sequential rounds per region: each round connects to all target domains in turn, so consecutive rounds for the same domain are separated by at least 12 hours; given that 91% of the observed TLSA records have a TTL of at most 12 hours, cached entries may expire between rounds. We exclude DNSSEC-invalid TLSA records, and any mismatch between a domain’s TLSA and its presented certificate is treated as a connection failure. Out of the 8,513 domains, 4,435 succeeded in all five attempts across both regions and both modes; these are used in our analysis. Of the remaining 4,078 domains: 2,434 (60%) fail even under STD-TLS; 1,539 (38%) present a certificate that does not match their TLSA record; the rest succeed under STD-TLS but fail under DANE-TLS.

Result analysis. Figure 5 shows page load times and paired DANE-induced overhead for the 4,435 domains. For page load times, each point represents the per-domain mean over five measurements. For overhead, we compute the difference between the five-run mean page load times of DANE-TLS and STD-TLS for each domain. The median overhead is 744.0 ms in US East and 42.4 ms in Germany, with p99 reaching 4,699.3 ms and 2,051.0 ms, respectively. The larger

US East overhead is consistent with the concentration of DANE-capable domains under Europe-based authoritative nameservers (Appendix E.2).

Comparison with prior work [40]. Our Germany measurements exhibit similar page load times with the Frankfurt measurements (under DNS caching) in Table II of [40]. Our Germany measurements report mean page load times of 1,863.5 ms for STD-TLS and 1,949.1 ms for DANE-TLS, with a mean paired per-domain overhead of 85.6 ms; Frankfurt measurements [40] report mean page load times of 1,973.9 ms without DANE and 2,104.6 ms with DANE, a difference of 130.7 ms. Nevertheless, these results are not directly equivalent due to the methodological differences (Section 2). In particular, the 130.7 ms overhead in [40] is the difference between the two reported mean page load times, whereas our 85.6 ms overhead is the mean of paired per-domain overheads.

Takeaway. Using Google DNS, DANE introduces a median overhead of 42 ms in Europe and 744 ms in US East, highlighting the impact of RTT on DANE overhead variability.

6 Discussion

Deployment considerations. Using a *full certificate* in a TLSA record (i.e., Selector 0 and Matching Type 0) can push the DNS response size above the UDP fragmentation threshold, triggering DNS-over-TCP fallback and increasing DNS latency by up to 2x. This effect becomes more pronounced when the client/resolver is far from the authoritative nameserver. In our setup, the 000 and 200 combinations, which embed Let's Encrypt's E7 intermediate certificate [26], consistently trigger fallback. Because Let's Encrypt accounts for over 50% of web certificates [14, 19], many real-world domains could encounter the same issue when using these combinations. 100 and 300 avoid fallback only because the leaf certificate in our experiment is small; larger certificates would trigger the same issue. TLSA with *full public key* (Selector 1 and Matching Type 0) are smaller today, but future larger keys, such as post-quantum cryptography algorithms (e.g., Dilithium [9], which is at least 1,312 bytes), may again exceed the threshold. Consistent with the DANE RFC [12], which marks Matching Type 0 as *NOT RECOMMENDED*, operators should avoid publishing full certificates via TLSA records to prevent TCP fallback-induced delays.

We also find that using a Let's Encrypt certificate chain does not introduce additional delay. Our web server serves the full chain in both STD-TLS and DANE-TLS modes, even though DANE-EE (i.e., Usage 3) allows serving only the leaf certificate. Additional experiments (Appendix D.2) where we served only the leaf certificate show virtually identical delays. This indicates that domains can support both standard TLS and DANE during a transition period by continuing to serve a conventional CA-based certificate chain (e.g., Let's Encrypt) while also publishing a TLSA record (e.g., DANE-EE), without incurring extra latency.

EDNS0. Advertising a larger EDNS0 buffer does not appear sufficient to avoid TCP fallback in our measurements: the 1,232-byte threshold reflects a recommended UDP response-size limit intended to prevent IP fragmentation [11, 16], and this operational limit can be applied even when the requester (e.g., a client) advertises a larger EDNS0 buffer. Consistent with this, for two full-certificate domains

(i.e., using TLSA 000) in our web-scale study, we confirmed that TLSA lookups through Google DNS trigger TCP fallback and return responses exceeding 1,700 bytes after TCP retry, even when the query advertises a larger EDNS0 buffer (Appendix F).

Measurement limitations. Our measurement setup uses Firefox in headless mode and Docker-based networking. Headless execution may differ from conventional interactive browsing in browser-level behavior, such as rendering and resource scheduling. Therefore, the absolute webpage load times measured in our setup may not exactly represent user-perceived performance in a conventional desktop browser. Docker networking may also introduce additional processing overhead through virtual network interfaces, bridges, and container-to-container forwarding. However, STD-TLS and DANE-TLS use the same browser, proxy, and Docker network topology, so much of this systematic overhead is common to both modes, although we do not assume that it cancels completely. Accordingly, our results should primarily be interpreted in terms of relative DANE-induced delay and its protocol-level causes, rather than as exact estimates of end-user QoE.

Future work. To reduce latency, one possible option is to deploy TLS DNSSEC Chain Extension [13], which embeds TLSA record and its full DNSSEC chain into TLS handshake, eliminating the need for separate DNSSEC/TLSA lookups. The extension is yet to be deployed and studied in practice.

7 Conclusion

We presented the first fine-grained analysis of DANE's impact on web performance. Using our DANE-for-the-web measurement framework, we conducted controlled experiments showing that delay is dominated by *DNS phase*, which is highly sensitive to network latency and caching behavior; when records are warm-cached, DANE adds virtually no overhead. We also find that those embedding full certificates trigger TCP fallback and substantially increase lookup delay. Our web-scale study quantifies the delay introduced by DANE on real-world domains. Finally, we provide guidelines for efficient DANE deployment.

Acknowledgments

We thank anonymous reviewers for their insightful and constructive suggestions and feedback. This work is supported by National Science Foundation CNS-2319421 and the Commonwealth Cyber Initiative. Additionally, this work used AWS via Cloudbank [33] through allocation CIS260181 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program [4], which is supported by U.S. National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296. This work was also supported by the research fund of Hanyang University (HY-2026-1165) and by the Ministry of Trade, Industry and Resources (MOTIR, Korea), under the project titled "An Unbreachable Multilayer AI-based Security Mechanism that Continuously Adapts and Evolves in Dynamic Conditions" (Project Number: RS-2025-02653102). The research leading to these results was made possible by OpenINTEL (<https://www.openintel.nl/>), a joint project of the University of Twente, SIDN, NLnet Labs, and SURF. Finally, the domain name data used in this research were obtained from the Internet.nl project and used with their permission.

References

- [1] APNIC. 2022. Looking at centrality in the DNS. <https://blog.apnic.net/2022/11/22/looking-at-centrality-in-the-dns/> (Accessed: Jun. 3, 2026).
- [2] Apple. 2026. Available trusted root certificates for Apple operating systems. <https://support.apple.com/en-us/103272> (Accessed: Jun. 3, 2026).
- [3] BBC. 2011. Fake DigiNotar web certificate risk to Iranians. <https://www.bbc.com/news/technology-14789763> (Accessed: Jun. 3, 2026).
- [4] Timothy J. Boerner, Stephen Deems, Thomas R. Furlani, Shelley L. Knuth, and John Towns. 2023. ACCESS: Advancing Innovation: NSF's Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support. In *Practice and Experience in Advanced Research Computing 2023: Computing for the Common Good* (Portland, OR, USA) (PEARC '23). Association for Computing Machinery, New York, NY, USA, 173–176. <https://doi.org/10.1145/3569951.3597559>
- [5] buffrr. 2026. Let's DANE. <https://github.com/buffrr/letsdane> (Accessed: Jun. 3, 2026).
- [6] Cisco. 2026. Umbrella Popularity List. <https://umbrella-static.s3-us-west-1.amazonaws.com/index.html> (Accessed: Apr. 29, 2026).
- [7] Cloudflare Radar. 2026. Domain Rankings. <https://radar.cloudflare.com/domains> (Accessed: Apr. 29, 2026).
- [8] Comodo. 2011. Comodo Fraud Incident. <https://www.comodo.com/Comodo-Fraud-Incident-2011-03-23.html> (Accessed: Jun. 3, 2026).
- [9] CRYSTALS. 2026. Dilithium. <https://pq-crystals.org/dilithium/> (Accessed: Jun. 3, 2026).
- [10] David Dittrich, Erin Kenneally, et al. 2012. *The Menlo Report: Ethical principles guiding information and communication technology research*. Technical Report. US Department of Homeland Security.
- [11] DNS flag day. 2020. DNS flag day. <https://www.dnsflagday.net/2020/> (Accessed: Jun. 3, 2026).
- [12] Viktor Dukhovni and Wes Hardaker. 2015. The DNS-Based Authentication of Named Entities (DANE) Protocol: Updates and Operational Guidance. RFC 7671. <https://doi.org/10.17487/RFC7671>
- [13] Viktor Dukhovni, Shumon Huque, Willem Toorop, Paul Wouters, and Melinda Shore. 2021. TLS DNSSEC Chain Extension. RFC 9102. <https://doi.org/10.17487/RFC9102>
- [14] Syed Muhammad Farhan and Taejoong Chung. 2023. Exploring the Evolution of TLS Certificates. In *Passive and Active Measurement: 24th International Conference, PAM 2023, Virtual Event, March 21–23, 2023, Proceedings*. Springer-Verlag, Berlin, Heidelberg, 71–84.
- [15] Tobias Fiebig and Anja Feldmann. 2025. 'How I learned to stop worrying and love IPv6': Measuring the Internet's Readiness for DNS over IPv6. In *Proceedings of the 2025 ACM Internet Measurement Conference*. 359–380.
- [16] Kazunori Fujiwara and Paul A. Vixie. 2025. IP Fragmentation Avoidance in DNS over UDP. RFC 9715. <https://doi.org/10.17487/RFC9715>
- [17] Paul E. Hoffman and Jakob Schlyter. 2012. The DNS-Based Authentication of Named Entities (DANE) Transport Layer Security (TLS) Protocol: TLSA. RFC 6698. <https://doi.org/10.17487/RFC6698>
- [18] Shumon Huque. 2015. Whither DANE? <https://blog.apnic.net/2019/07/05/whither-dane/> (Accessed: Jun. 3, 2026).
- [19] Internet Security Research Group (ISRG). 2024. 2024 Annual Report. <https://www.abetterinternet.org/documents/2024-ISRG-Annual-Report.pdf> (Accessed: Jun. 3, 2026).
- [20] internet.nl. 2026. Hall of Fame - Websites. <https://en.internet.nl/halloffame/web/> (Accessed: Jun. 3, 2026).
- [21] ISC. 2026. BIND9 Docker. <https://hub.docker.com/r/internetsystemsconsortium/bind9> (Accessed: Jun. 3, 2026).
- [22] Liz Izhikevich, Gautam Akiwate, Briana Berger, Spencer Drakontaidis, Anna Ascherman, Paul Pearce, David Adrian, and Zakir Durumeric. 2022. ZDNS: a fast DNS toolkit for internet measurement. In *Proceedings of the 22nd ACM Internet Measurement Conference (Nice, France) (IMC '22)*. Association for Computing Machinery, New York, NY, USA, 33–43.
- [23] Adam Langley. 2015. Why not DANE in browsers. <https://www.imperialviolet.org/2015/01/17/notdane.html> (Accessed: Jun. 3, 2026).
- [24] Hyeonmin Lee, Md. Ishtiaq Ashiq, Moritz Müller, Roland van Rijswijk-Deij, Taekyoung "Ted" Kwon, and Taejoong Chung. 2022. Under the Hood of DANE Mismanagement in SMTP. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 1–16.
- [25] Hyeonmin Lee, Aniketh Girish, Roland van Rijswijk-Deij, Taekyoung "Ted" Kwon, and Taejoong Chung. 2020. A longitudinal and comprehensive study of the DANE ecosystem in email. In *Proceedings of the 29th USENIX Conference on Security Symposium (SEC'20)*. USENIX Association, USA, Article 35, 18 pages.
- [26] Let's Encrypt. 2026. Chain of Trust - Subordinate (Intermediate) CA. <https://letsencrypt.org/certificates/#subordinate-intermediate-cas> (Accessed: Jun. 3, 2026).
- [27] John Leyden. 2013. French gov used fake Google certificate to read its workers' traffic. The Register. https://www.theregister.com/2013/12/10/french_gov_dodgy_ssl_cert_reprimand/ (Accessed: Jun. 3, 2026).
- [28] Majestic. 2026. Majestic Million. <https://majestic.com/reports/majestic-million> (Accessed: Apr. 29, 2026).
- [29] Neil McAllister. 2015. Mozilla piles on China's SSL cert overlord: We don't trust you either. The Register. https://www.theregister.com/2015/04/02/mozilla_revokes_cnnic_cert_trust/ (Accessed: Jun. 3, 2026).
- [30] Microsoft. 2026. Release notes - Microsoft Trusted Root Certificate Program. <https://learn.microsoft.com/en-us/security/trusted-root/release-notes> (Accessed: Jun. 3, 2026).
- [31] Mozilla. 2026. Firefox 140.0.2. https://ftp.mozilla.org/pub/firefox/releases/140.0.2/linux-x86_64/ (Accessed: Jun. 3, 2026).
- [32] Mozilla Wiki. 2026. Mozilla's CA Certificate Program. <https://wiki.mozilla.org/CA> (Accessed: Jun. 3, 2026).
- [33] Michael Norman, Vince Kellen, Shava Smullen, Brian DeMeulle, Shawn Strande, Ed Lazowska, Naomi Alterman, Rob Fatland, Sarah Stone, Amanda Tan, Katherine Yelick, Eric Van Dusen, and James Mitchell. 2021. CloudBank: Managed Services to Simplify Cloud Access for Computer Science Research and Education. In *Practice and Experience in Advanced Research Computing 2021: Evolution Across All Dimensions* (Boston, MA, USA) (PEARC '21). Association for Computing Machinery, New York, NY, USA, Article 45, 4 pages. <https://doi.org/10.1145/3437359.3465586>
- [34] OpenINTEL. 2026. Forward DNS - Top Lists. <https://openintel.nl/data/forward-dns/top-lists/> (Accessed: Apr. 29, 2026).
- [35] Victor Le Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Maciej Koczyński, and Wouter Joosen. 2026. Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation. <https://tranco-list.eu/> (Accessed: Apr. 29, 2026).
- [36] Scott Rose, Matt Larson, Dan Massey, Rob Austein, and Roy Arends. 2005. DNS Security Introduction and Requirements. RFC 4033. <https://doi.org/10.17487/RFC4033>
- [37] Scott Rose, Matt Larson, Dan Massey, Rob Austein, and Roy Arends. 2005. Protocol Modifications for the DNS Security Extensions. RFC 4035. <https://doi.org/10.17487/RFC4035>
- [38] Scott Rose, Matt Larson, Dan Massey, Rob Austein, and Roy Arends. 2005. Resource Records for the DNS Security Extensions. RFC 4034. <https://doi.org/10.17487/RFC4034>
- [39] Roland van Rijswijk-Deij, Mattijs Jonker, Anna Sperotto, and Aiko Pras. 2016. A High-Performance, Scalable Infrastructure for Large-Scale Active DNS Measurements. *IEEE Journal on Selected Areas in Communications* 34, 6 (2016), 1877–1888. <https://doi.org/10.1109/JSAC.2016.2558918>
- [40] Kota Yagi, Katsuki Isobe, Daishi Kondo, and Hideki Tode. 2024. Impact of DANE on Webpage Load Time. In *2024 20th International Conference on Network and Service Management (CNSM)*. 1–9.
- [41] Liang Zhu, Duane Wessels, Allison Mankin, and John Heidemann. 2015. Measuring DANE TLSA Deployment. In *Traffic Monitoring and Analysis*, Moritz Steiner, Pere Barlet-Ros, and Olivier Bonaventure (Eds.). Springer International Publishing, Cham, 219–232.
- [42] Ólafur Guðmundsson and Brian Wellington. 2003. Redefinition of DNS Authenticated Data (AD) bit. RFC 3655. <https://doi.org/10.17487/RFC3655>

A Ethics

Our study was conducted in strict accordance with established research ethics and the Menlo Report principles [10].

Stakeholders and ethical impact. Our research may affect several stakeholders, including websites that publish TLSA records and the broader Internet community. Consistent with the Menlo Report principles [10], we carefully designed our experiments to avoid harm and privacy risks, and to serve the public interest by improving the understanding of DANE deployment and its performance implications.

Data collection and privacy. All traffic in controlled experiments was confined to our own test infrastructure. For web-scale measurements, we primarily relied on the publicly available OpenINTEL dataset [34] and the domain list provided by Internet.nl, both of which have well-defined measurement objectives, methodologies, and issue-resolution processes. Using these datasets, we sent one-time TLSA record queries only to domains that publish DNSKEY records. Among them, for domains with TLSA records, our web-scale measurement involved HTTPS connections (and related



Figure 6: Example TLSA record for HTTPS on port 443.

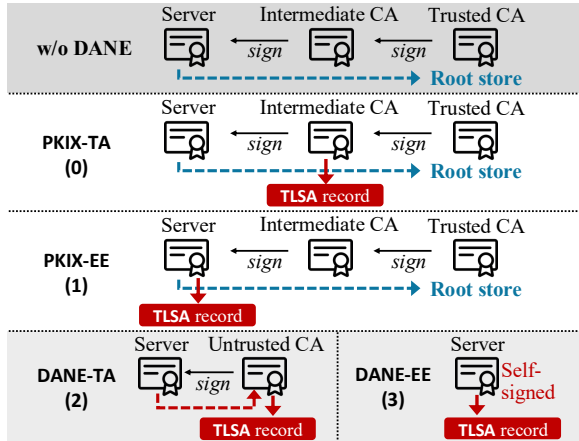


Figure 7: Comparison of Certificate Usage types. Dotted blue lines show traditional CA-based certificate validation using root stores, while red lines show DANE-based validation with TLSA records. In the PKIX-TA example, an intermediate CA certificate is used as TLSA.

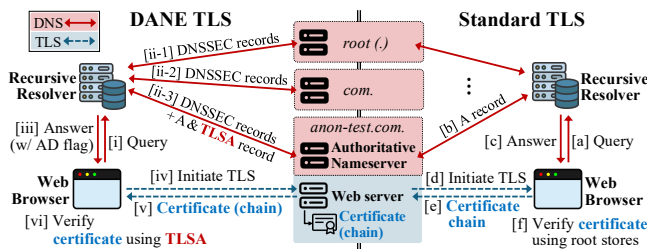


Figure 8: DNS and TLS handshake flows: with DANE (DANE TLS) and without DANE (Standard TLS).

DNS queries) to publicly accessible servers only. To avoid placing excessive load on target domains, we limited the number of connections to between 20 and 40 per domain. The baseline number of connections is 20 (2 measurement regions $\times$ 2 validation modes $\times$ 5 attempts), and the upper bound reflects one retry for failed attempts; here, the validation modes are STD-TLS and DANE-TLS. We also inserted a minimum 2 seconds pause between attempts to avoid overloading Google Public DNS. We did not collect, store, or analyze any Personally Identifiable Information (PII), authentication credentials, or cookies.

B Background: DANE and DNSSEC

TLSA record. Figure 6 shows an example TLSA record for HTTPS. Next, Figure 7 illustrates how certificate validation differs across Certificate Usage types during the *TLS phase*. These mechanisms govern which certificates must be transferred and how the client verifies them:

- **Standard TLS (w/o DANE):** A client validates the server certificate by constructing a chain from the leaf certificate up to a trusted root CA. Certificate verification relies on the root store bundled with the client (e.g., web browsers [2, 30, 32]) or Operating System (OS).
- **PKIX-TA (0):** The server publishes its intermediate (or root) certificate as the TLSA record. The server must provide the certificate chain during the TLS handshake. Upon receiving the chain, the client performs two checks: (i) it validates the chain using the root store, and (ii) it compares the intermediate certificate in the chain with TLSA fetched during the *DNS phase*.
PKIX-EE (1): This usage is similar to PKIX-TA, but the TLSA record contains the server's leaf certificate. The certificate validation remains the same as in PKIX-TA.
- **DANE-TA (2):** The server uses a certificate issued by a CA that is not included in the client's root store; e.g., an enterprise/organizational Trust Anchor (TA). The server publishes the CA/TA certificate as the TLSA record. To complete the TLS handshake, the server provides the leaf and CA/TA certificate, depending on the Matching Type of the TLSA record; if the TLSA record already includes the full CA certificate (i.e., a full-certificate matching type), the server may omit sending it in the TLS handshake. The client validates the chain using the certificate delivered by the server and then verifies that the CA/TA certificate matches the TLSA record.
- **DANE-EE (3):** The server publishes its leaf certificate (possibly self-signed) as the TLSA record, removing the need for chain validation. During the TLS handshake, the server presents only the leaf certificate, and the client checks whether it matches the value in the TLSA records.

DNS Security Extensions (DNSSEC). DNSSEC [36–38] guarantees the authenticity and integrity of DNS records. Since the original DNS was designed without any security features, it is vulnerable to attacks such as cache poisoning. As a solution, DNSSEC was proposed about two decades ago. To support DNSSEC, a domain (e.g., `anon-domain1.com`) must publish three types of records: DNSKEY, DS, and RRSIG. A DNSKEY record contains the public key (corresponding to the private key used for signing). The public key is uploaded to the parent zone (e.g., `.com`) in the form of a DS record to establish a chain of trust. Each DNS record served by the domain is then signed (with the private key), and the signature is stored as an RRSIG record. Through the trust chain formed from the root zone to the end-domain, the authenticity and integrity of DNS records are verified.

C Overhead of Let's DANE proxy

Figure 9 shows the additional delays introduced by Let’s DANE proxy compared to the baseline case, where the browser connects directly to the web server. Despite the difference in absolute latency across regions (due to RTT), the proxy adds less than 50 ms overhead across configurations.

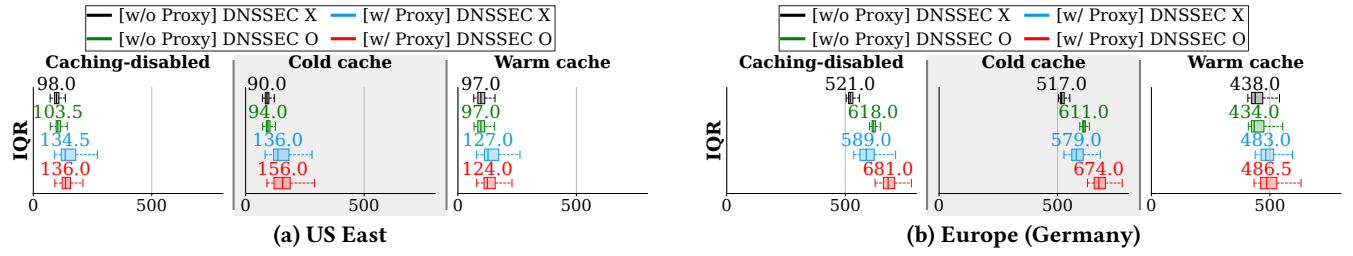


Figure 9: Additional delays introduced by the DANE proxy (w/ Proxy) compared to the baseline (w/o Proxy). The upper plots show CDFs, and the lower plots show interquartile range (IQR) box plots with median values.

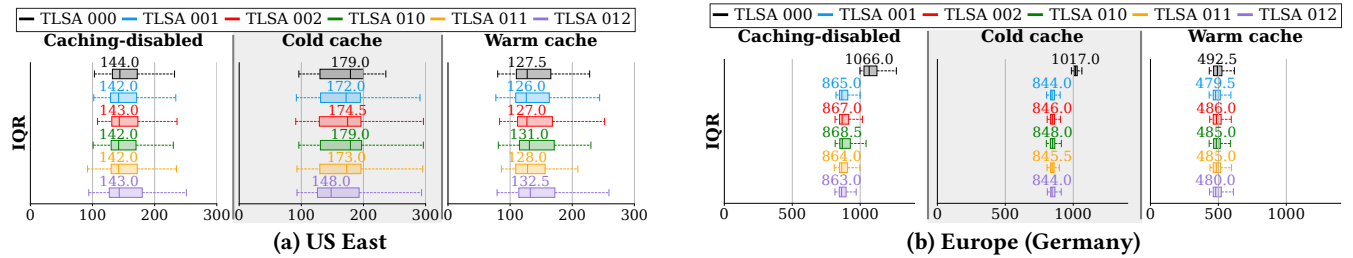


Figure 10: Page load time (ms) for Certificate Usage 0.

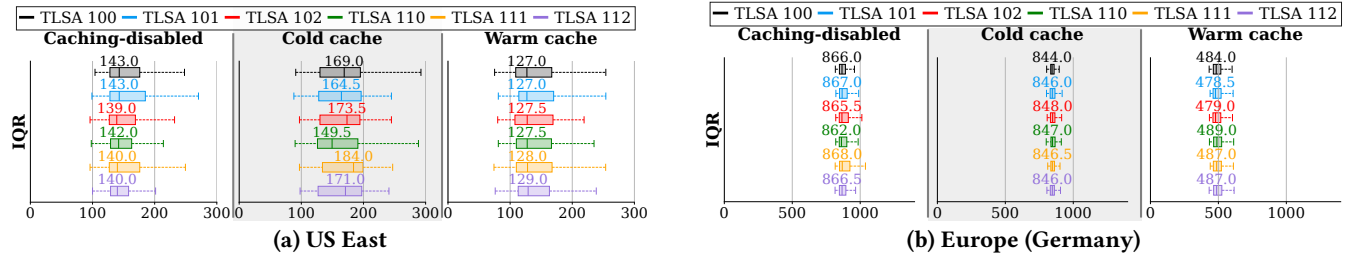


Figure 11: Page load time (ms) for Certificate Usage 1.

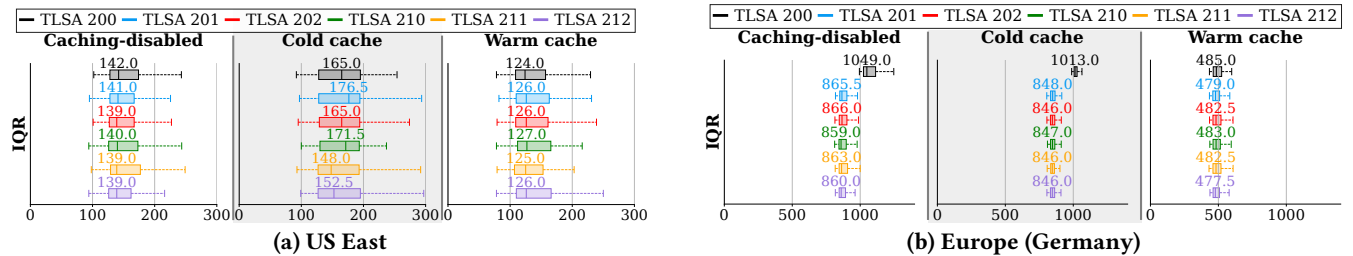


Figure 12: Page load time (ms) for Certificate Usage 2.

D Controlled Experiment: Supplementary

D.1 TLSA Parameter Combination - All

For completeness, we provide the full set of plots for all parameter combinations. The complete page load time results for Certificate Usage 0–3 are shown in Figure 10, Figure 11, Figure 12, and Figure 13. We also provide the full detailed delay plots in Figure 15, Figure 16, Figure 17, and Figure 18.

Fixing Selector and Matching Type to 1 shows that all four Certificate Usage types exhibit nearly identical delays. In US

East, page load times differ only marginally across Usage types under all cache settings. The detailed phase breakdown shows the same pattern: DANE-EE (3) is only about 3 ms faster than the others in the *DNS+TLS phase*, primarily due to its shorter *TLS phase* (i.e., due to differences in the certificate validation mechanism), while the *DNS phase* remains nearly identical across Usage types. Germany shows the same relative pattern with proportionally larger absolute delays, consistent with its higher RTT.

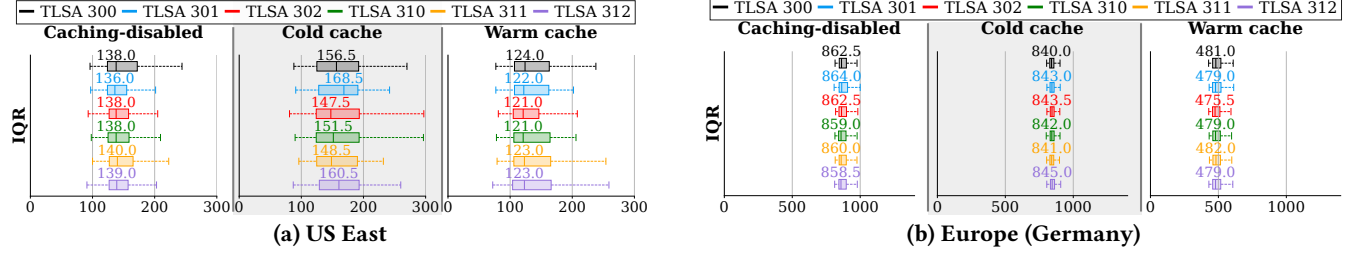


Figure 13: Page load time (ms) for Certificate Usage 3.

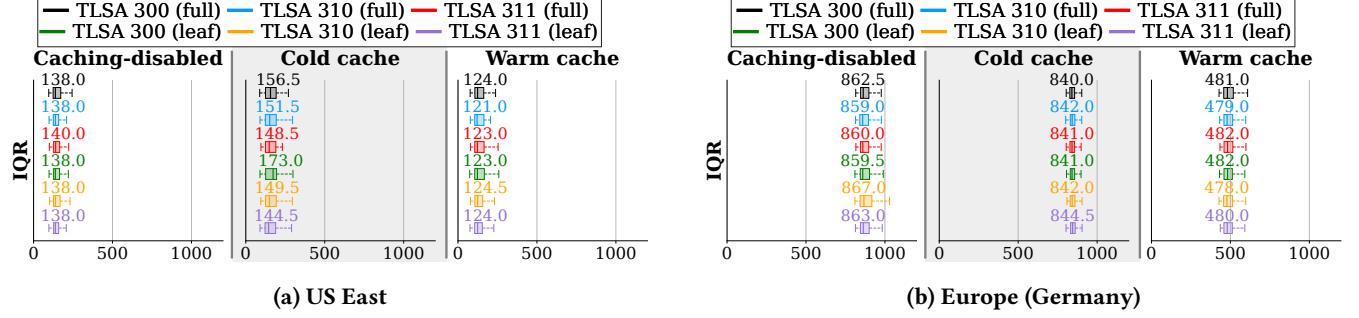


Figure 14: Page load times (ms) for full certificate-chain vs. leaf-only configurations. Certificate Usage is fixed to 3, and only TLSA 300, 310, and 311 are shown, as all other parameter combinations exhibit similar trends.

Table 1: TLSA parameter combinations used by domains in the web-scale experiment. Note that the ratios sum to more than 100% because a single domain may publish multiple TLSA with different combinations.

Certificate Usage	Selector	Matching Type	# Domain	Ratio
0 (PKIX-TA)	0	0	15	0.2%
		1	41	0.5%
		2	3	0.0%
	1	0	-	-
		1	287	3.4%
		2	8	0.1%
1 (PKIX-EE)	0	0	-	-
		1	60	0.7%
		2	16	0.2%
	1	0	1	0.0%
		1	197	2.3%
		2	165	1.9%
2 (DANE-TA)	0	0	-	-
		1	95	1.1%
		2	34	0.4%
	1	0	-	-
		1	1,630	19.1%
		2	39	0.5%
3 (DANE-EE)	0	0	1	0.0%
		1	3,220	37.8%
		2	77	0.9%
	1	0	15	0.2%
		1	3,656	42.9%
		2	382	4.5%
Total			8,513	-

‘-’: No domains use this combination.

Within each Certificate Usage type, varying Selector and Matching Type generally produces no meaningful differences. The only notable exceptions are the TLSA 000 and 200 combinations, as discussed in Section 4.1.2. The TLSA 100 and TLSA 300 combinations also embed full certificates, but their certificates (i.e., the leaf certificate) are smaller (1,096 bytes in our setup), keeping the

Table 2: The top seven Top-Level Domains (TLDs) of authoritative nameservers serving 4,435 domains in our web-scale analysis. A domain may use multiple nameservers, so the ratios can exceed 100%.

TLD of Nameserver	# Domain	Ratio
.nl (Netherlands)	1,854	41.8%
.eu (Europe)	1,587	35.8%
.be (Belgium)	1,263	28.5%
.net	498	11.2%
.com	497	11.2%
.org	153	3.4%
.de (Germany)	81	1.8%

response below the fragmentation threshold; thus no TCP fallback occurs. However, if these certificates grow beyond 1,232 bytes, they may similarly trigger TCP fallback and incur higher delays.

D.2 DANE and Certificate Chain Length

With DANE-EE usage, a server may serve only a leaf certificate (possibly self-signed), and the client needs only to compare it with the TLSA record. In our main setup, however, we use a Let’s Encrypt certificate chain (intermediate and leaf) so that both STD-TLS and DANE-TLS measurements are supported; STD-TLS requires chain validation via root stores, whereas DANE-EE does not. To evaluate whether serving a full chain affects performance, we compare delays between two configurations: (1) serving the certificate chain and (2) serving only the leaf certificate.

Figure 14 presents the page load times for three representative parameter combinations (300, 310, 311). Across both regions, we

observe no measurable difference between the two configurations, and all remaining combinations exhibit the same pattern.

E Web-scale Measurement: Supplementary

E.1 TLSA Parameter Statistics

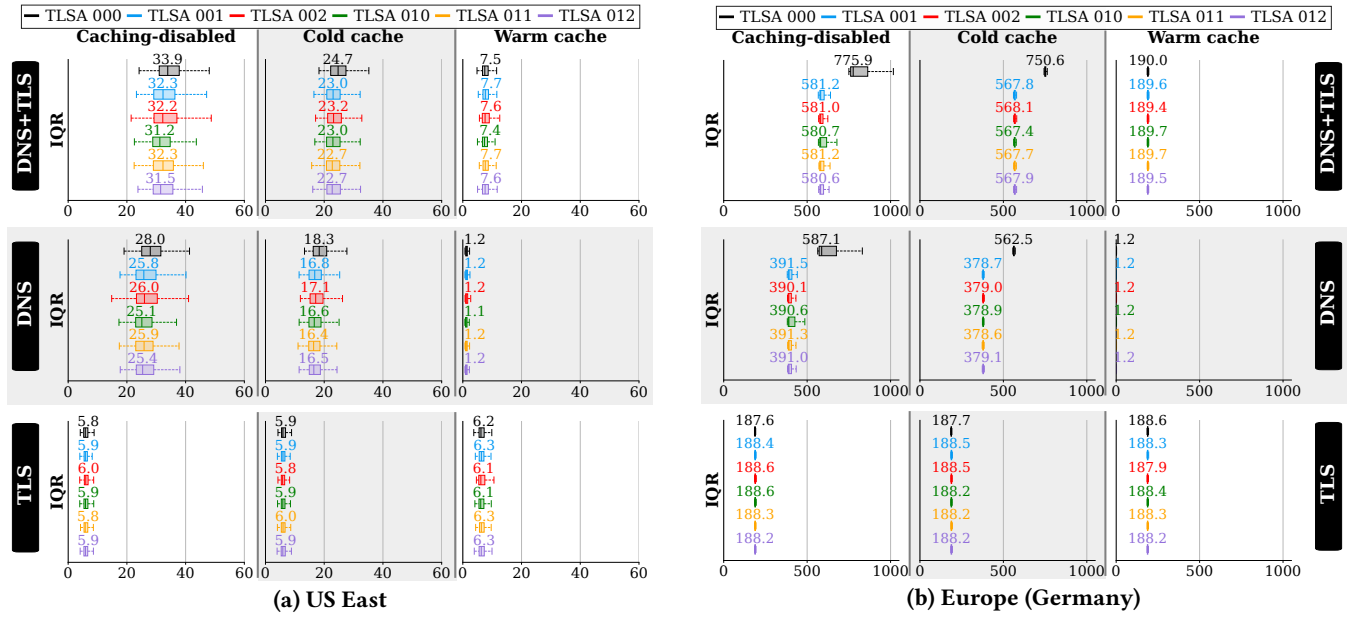
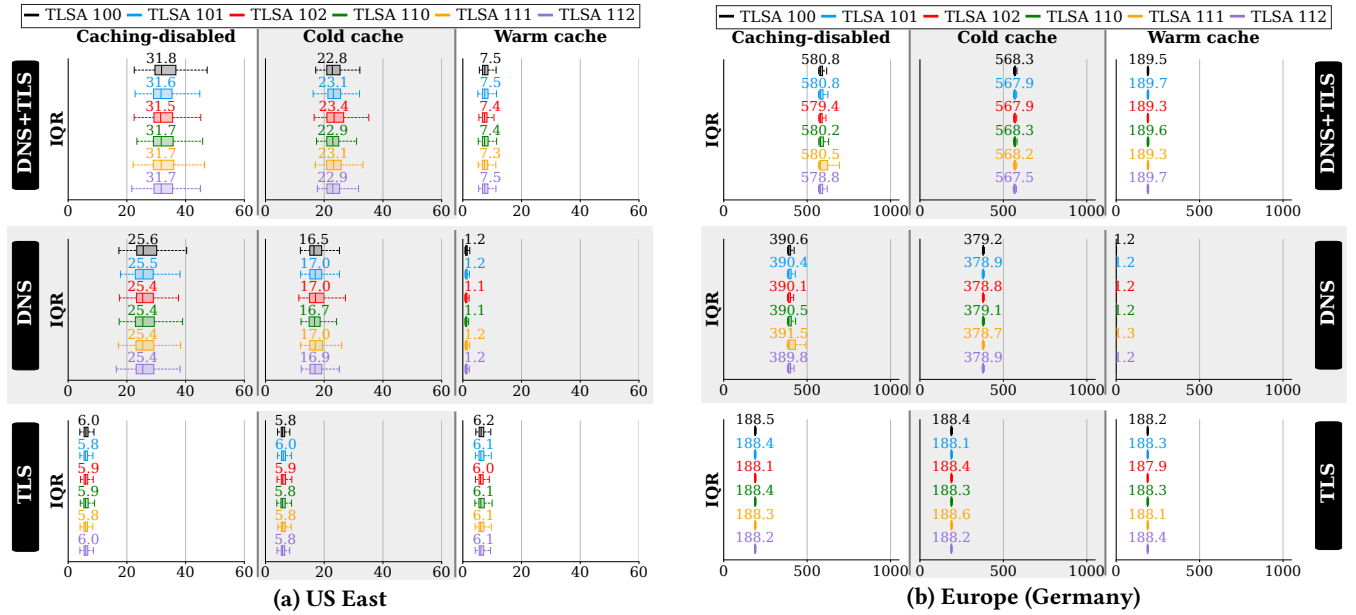
Table 1 summarizes the TLSA parameter combinations used by the 8,513 target domains in our web-scale measurement. The most common combinations are 311 (42.9%) and 301 (37.8%). We also observe that 15 domains (0.2%) publish TLSA 000 records. Among these, 12 contain invalid TLSA values: although marked as Matching Type 0, they actually publish SHA-256 digests. Two domains indeed publish full-certificate values in their TLSA records.

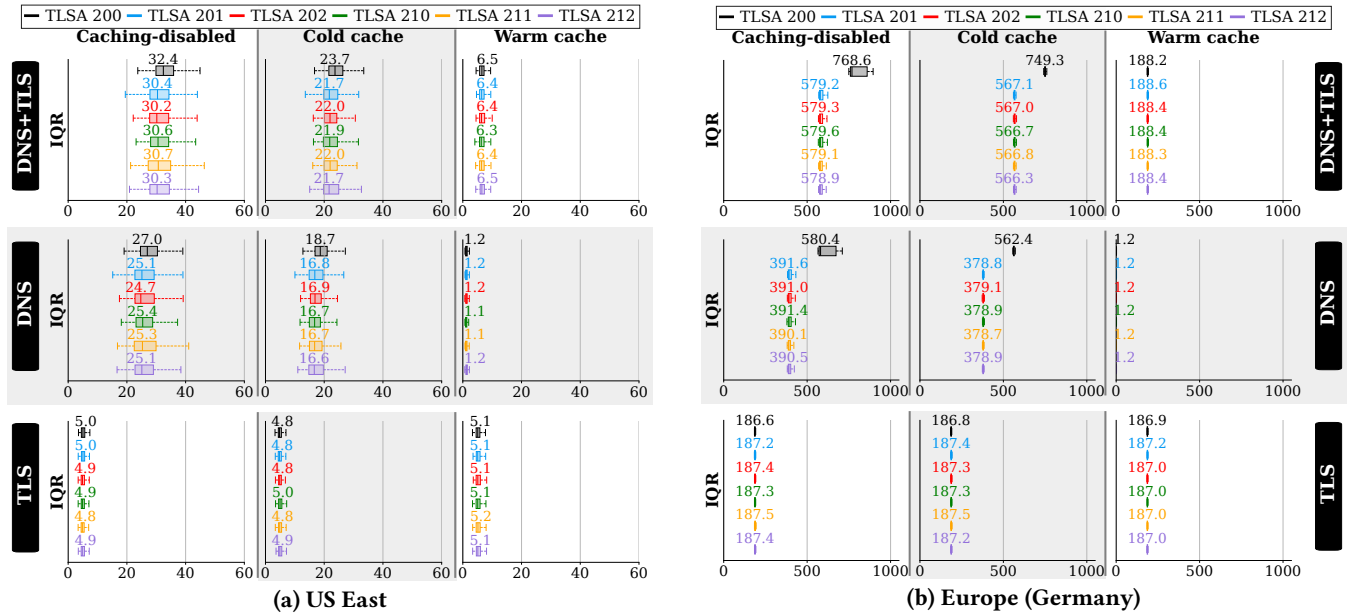
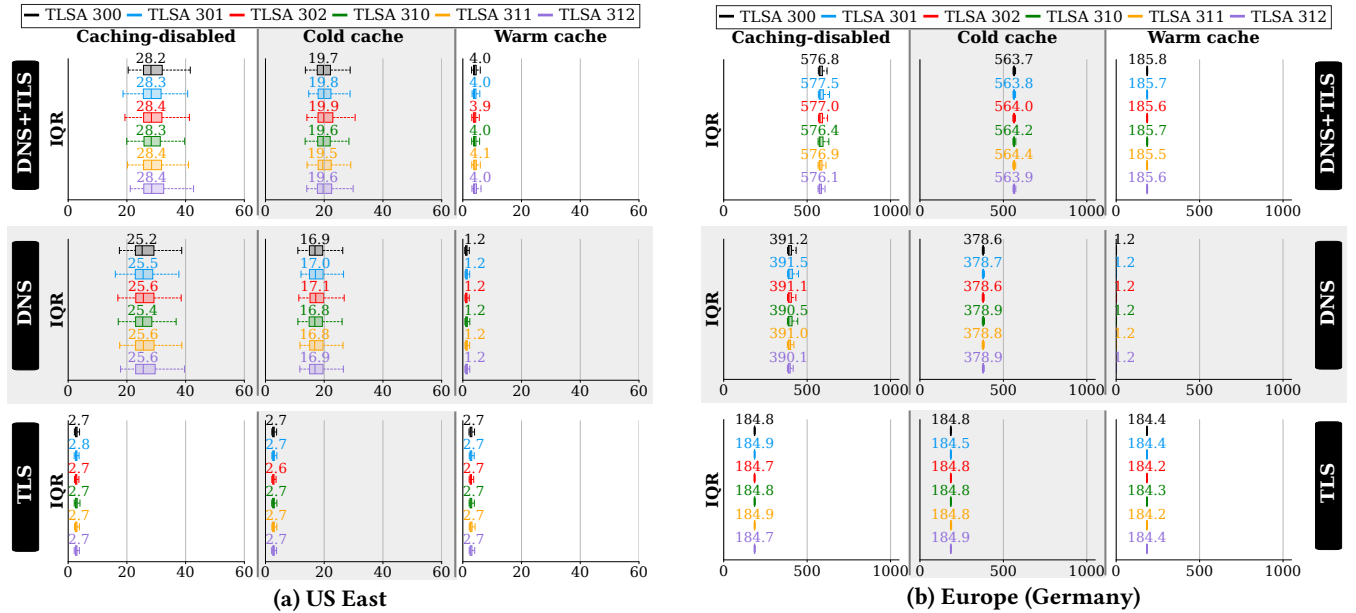
E.2 TLD Distribution of Nameservers

Table 2 shows the top seven Top-Level Domains (TLDs) of authoritative nameservers serving the domains in our web-scale analysis. The top three TLDs, covering more than 56% of domains, are all based in Europe. This geographic concentration explains why page load times measured from Germany are noticeably lower than those from US East.

F EDNS0 Effect

A natural question is whether advertising a larger EDNS0 UDP buffer could avoid the DNS-over-TCP fallback we observe for oversized TLSA responses. In our setup, both the DANE proxy and the recursive resolver advertise a 4,096-byte EDNS0 buffer, yet TCP fallback still occurs. A likely explanation is that, in practice, the effective limit is not only the client-advertised EDNS0 buffer size, but also the UDP response-size limit adopted to avoid IP fragmentation. Since DNS Flag Day 2020 [11] and RFC 9715 [16] recommend limiting UDP responses around 1,232 bytes, servers and resolvers following this guidance may set the TC bit for larger responses even when the client advertises a larger buffer. A larger EDNS0 buffer may therefore not be sufficient to avoid such fallback. In addition, for two domains with full-certificate TLSA records (i.e., TLSA 000), we verified that DNSSEC-enabled TLSA lookups through Google DNS return responses exceeding 1,700 bytes and trigger TCP fallback, even when using a 4,096-byte advertised EDNS0 buffer. Consistent with this, a recent study [15] reports that, for DNSSEC-signed zones, TCP fallbacks remain frequent even with a 4,096-byte EDNS0 buffer. Overall, our observations suggest that avoiding oversized (full-certificate) TLSA records is a more robust approach than relaxing fragmentation-avoidance settings.

Figure 15: Detailed delays (*DNS+TLS*, *DNS*, and *TLS*) for Certificate Usage 0 across two deployment regions.Figure 16: Detailed delays (*DNS+TLS*, *DNS*, and *TLS*) for Certificate Usage 1 across two deployment regions.

Figure 17: Detailed delays (*DNS+TLS*, *DNS*, and *TLS*) for Certificate Usage 2 across two deployment regions.Figure 18: Detailed delays (*DNS+TLS*, *DNS*, and *TLS*) for Certificate Usage 3 across two deployment regions.